

ANEXO B – Documentação de uso do PFFc

Neste anexo é apresentada a documentação detalhada dos módulos da biblioteca. O texto descreve as funcionalidades de cada classe e função, expondo os parâmetros de entrada necessários, os dados de saída, as condições de contorno e as dependências entre as classes. Dessa forma, objetiva-se fornecer embasamento suficiente para que o usuário utilize a ferramenta de forma plena, sem a necessidade de inspecionar os códigos para compreendê-lo.

1. Visão Geral dos Módulos

O sistema é composto por 7 módulos Python independentes. A tabela abaixo resume cada arquivo, sua responsabilidade e as classes exportadas.

Arquivo	Responsabilidade	Classes
propriedades.py	Geometria dos perfis (seção transversal)	PerfilBase, USimples, UEnrijecido, Cartola
tracao.py	Capacidade de tração	CapacidadeTracao
compressao_mle.py	Capacidade de compressão (MLE)	CompressaoMLE
flexao_mle.py	Capacidade de flexão (MLE)	FlexaoMLE
cortante.py	Resistência ao esforço cortante	ResistenciaCortante
estabilidade_global.py	Estabilidade global (flambagem)	EstabilidadeGlobal
combinados.py	Esforços combinados	EsforcosCombinados

2. Dependências Externas

Todos os módulos utilizam exclusivamente a biblioteca padrão do Python. Não há dependências externas de terceiros, como numpy, pandas, scipy, etc.

Módulo	Origem	Uso
math	Python stdlib	Funções trigonométricas (sin, radians), sqrt, pi, log – utilizadas em propriedades.py, compressao_mle.py, flexao_mle.py e estabilidade_global.py.
tkinter	Python stdlib	Biblioteca padrão do Python para criar Interfaces Gráficas de Usuário (GUI) – utilizada no arquivo orquestrador ppfc.py.

3. Módulo: propriedades.py

Contém as classes responsáveis por calcular as propriedades geométricas da seção transversal para três tipologias de perfis formados a frio. Todas as dimensões de entrada são em milímetros (mm); as propriedades geométricas resultantes também são armazenadas em mm e mm². As classes desse módulo devem ser instanciadas antes de qualquer outro módulo, uma vez que forcem as informações básicas para qualquer cálculo seguinte.

3.1 PerfilBase (classe abstrata)

Classe base da hierarquia, deve ser instanciada antes das demais. Define Propriedades geométricas comuns a todos os perfis (raio interno, raio médio e comprimentos de curva) e a interface que as subclasses devem implementar.

Construtor

```
PerfilBase(t: float, fy: float, fu: float = None)
```

Parâmetros importados

Parâmetro	Tipo	Unidade	Descrição
t	float	mm	Espessura da chapa.
fy	float	MPa	Tensão de escoamento do aço.
fu	float	MPa	Tensão de ruptura. Padrão: $1,15 \times fy$.

Parâmetros calculados

Parâmetro	Tipo	Unidade	Descrição
ri	float	mm	Raio interno de dobramento. $ri = t$ se $t \leq 6,3$ mm; $ri = 1,5t$ caso contrário.
rm	float	mm	Raio médio de dobramento. $rm = ri + 0,5t$.
u1	float	mm	Comprimento de arco de 90° (entre tangentes): $1,571 \times rm$.
u2	float	mm	Comprimento de arco de 45°: $0,785 \times rm$.
aviso	list[str]	—	Lista de mensagens de aviso emitidas durante a validação das relações b/t.

Funções

Função	Descrição	Retorno
<code>_calcular_parametros_comuns(self)</code>	Calcula os parâmetros comuns a todas seções.	None
<code>validar_dimensoes_basicas(self)</code>	Verifica se os trechos planos calculados (<code>a_plana</code> , <code>b_plana</code>) são positivos. Lança <code>ValueError</code> em caso de geometria inválida.	None

3.2 USimples

Construtor

```
USimples(t, fy, bw, bf, fu=None)
```

Parâmetros importados

Parâmetro	Tipo	Unidade	Descrição
t	float	mm	Espessura da chapa.
fy	float	MPa	Tensão de escoamento.
bw	float	mm	Altura total do perfil ($0 < bw \leq 2000$ mm).
bf	float	mm	Largura total da mesa ($0 < bf \leq 600$ mm).
fu	float	MPa	Tensão de ruptura. Opcional.

Parâmetros calculados

Parâmetro	Tipo	Unidade	Descrição
A	float	mm ²	Área bruta da seção transversal.
xg	float	mm	Coordenada X do centroide (medida desde a face aberta).
yg	float	mm	Coordenada Y do centroide (= bw/2, por simetria horizontal).
x0	float	mm	Coordenada X do centro de cisalhamento.
y0	float	mm	Coordenada Y do centro de cisalhamento (= 0, por simetria).
Ix	float	mm ⁴	Momento de inércia em relação ao eixo X (baricêntrico horizontal).
Iy	float	mm ⁴	Momento de inércia em relação ao eixo Y (baricêntrico vertical).
Ixy	float	mm ⁴	Produto de inércia (= 0 para seção com simetria em X).
It	float	mm ⁴	Constante de torção de Saint-Venant.
Cw	float	mm ⁶	Constante de empenamento da seção.
rx	float	mm	Raio de giração em X.
ry	float	mm	Raio de giração em Y.
r0	float	mm	Raio de giração polar (inclui excentricidade do centro de cisalhamento).
a_plana	float	mm	Comprimento plano da alma.
b_plana	float	mm	Comprimento plano da mesa.

Parâmetro	Tipo	Unidade	Descrição
angulo	float	graus	Ângulo entre alma e mesa (90°).

3.3 UEnrijecido

Construtor

```
UEnrijecido(t, fy, bw, bf, d, fu=None)
```

Parâmetros importados

Parâmetro	Tipo	Unidade	Descrição
t	float	mm	Espessura da chapa.
fy	float	MPa	Tensão de escoamento.
bw	float	mm	Altura total do perfil ($0 < bw \leq 2500$ mm).
bf	float	mm	Largura total da mesa ($0 < bf \leq 600$ mm).
d	float	mm	Altura do enrijecedor de borda ($0 < d \leq 600$ mm).
fu	float	MPa	Tensão de ruptura. Opcional.

Parâmetros calculados

Calcula todos os Parâmetros de USimples e acrescenta:

Parâmetro	Tipo	Unidade	Descrição
c_plana	float	mm	Comprimento plano do enrijecedor de borda.
c_total	float	mm	Altura total do enrijecedor (= d).

3.4 Cartola

Construtor

```
Cartola(t, fy, bw, bf, d, fu=None)
```

Parâmetros importados

Parâmetro	Tipo	Unidade	Descrição
t	float	mm	Espessura da chapa.
fy	float	MPa	Tensão de escoamento.
bw	float	mm	Altura total das almas ($0 < bw \leq 600$ mm).

Parâmetro	Tipo	Unidade	Descrição
bf	float	mm	Largura total da mesa superior ($0 < bf \leq 5000$ mm).
d	float	mm	Altura dos enrijecedores inferiores ($0 < d \leq 300$ mm).
fu	float	MPa	Tensão de ruptura. Opcional.

Parâmetros calculados

Calcula os mesmos Parâmetros de UEnrijecido. As coordenadas do centroide (xg, yg) e do centro de cisalhamento (x0, y0) são calculadas considerando a configuração específica da seção Cartola.

4. Módulo: estabilidade_global.py

Calcula as cargas críticas de flambagem elástica e o fator de redução χ (chi) para estabilidade global. Deve ser instanciado antes de CompressaoMLE e FlexaoMLE, pois fornece χ e cargas críticas para esses módulos.

Construtor

```
EstabilidadeGlobal(perfil_obj, lx: float, ly: float, lz: float)
```

Parâmetros importados

Parâmetro	Tipo	Unidade	Descrição
perfil_obj	objeto PerfilBase	–	Instância do perfil com propriedades geométricas calculadas: USimples(t, fy, bw, bf, fu=None), UEnrijecido(t, fy, bw, bf, d, fu=None) OU Cartola(t, fy, bw, bf, d, fu=None).
lx	float	mm	Comprimento efetivo de flambagem em torno do eixo X ($K_x \cdot L$).
ly	float	mm	Comprimento efetivo de flambagem em torno do eixo Y ($K_y \cdot L$).
lz	float	mm	Comprimento efetivo de flambagem por torção ($K_z \cdot L$).

NOTA: run_analysis() é chamado automaticamente no __init__ e lança ValueError se a relação L/r máxima for ≥ 200 .

Parâmetros calculados

Parâmetro	Tipo	Unidade	Descrição
Nex	float	N·mm	Carga crítica de flambagem por flexão em torno do eixo X (Euler).

Parâmetro	Tipo	Unidade	Descrição
Ney	float	N·mm	Carga crítica de flambagem por flexão em torno do eixo Y (Euler).
Nez1	float	N·mm	Carga crítica de flambagem por torção pura.
Nez2	float	N·mm	Carga crítica de flambagem por flexo-torção (apenas U simples e U enrijecido).
Nez3	float	N·mm	Carga crítica de flambagem por flexo-torção (apenas Cartola).
Ne_min	float	N·mm	Menor carga crítica entre os modos governantes.
lambda0	float	–	Índice de esbeltez reduzido.
chi_global	float	–	Fator de redução da resistência à compressão global ($0 < \chi \leq 1,0$).

Métodos

Função	Descrição	Retorno
<code>run_analysis(self)</code>	Calcula todas as cargas críticas e <code>chi_global</code> . Chamado automaticamente no construtor.	None
<code>get_memoria(self)</code>	Retorna dicionário com cargas críticas (em kN) e parâmetros de <code>chi</code> para uso em relatórios.	dict

Estrutura do dicionário retornado por `get_memoria()`

```
{
  "cargas_criticas": {
    "Nex", "Ney", "Nez", "Nexz"/"Neyz", "Ne_min" [kN]},
  "parametros_chi": {
    "lambda_0": float, "chi_global": float}
}
```

5. Módulo: `compressao_mle.py`

Implementa o Método da Largura Efetiva (MLE) para compressão uniforme na seção, calculando a área efetiva e a resistência à compressão (N_{rd}) conforme a NBR 14762:2010. Requer uma instância de `EstabilidadeGlobal` previamente calculada.

Construtor

```
CompressaoMLE(perfil_geom, chi: float = 1.0)
```

Parâmetros importados

Parâmetro	Tipo	Unidade	Descrição
<code>perfil_geom</code>	objeto <code>PerfilBase</code>	–	Instância do perfil com propriedades calculadas.

Parâmetro	Tipo	Unidade	Descrição
chi	float	–	Fator chi_global proveniente de EstabilidadeGlobal. Padrão: 1,0 (sem flambagem global).

Parâmetros calculados

Parâmetro	Tipo	Unidade	Descrição
chi_global	float	–	Fator de redução global recebido no construtor.
tensao	float	kN/cm ²	Tensão atuante de compressão: $\chi \times f_y$.
aefetiva	float	cm ²	Área efetiva calculada após aplicação do MLE.
nrd	float	kN	Resistência à compressão de cálculo.

Métodos

Função	Descrição	Retorno
<code>get_dimplanas(self)</code>	Extrai as dimensões planas (a, b, c) do objeto perfil e armazena em self.dimplanas. Retorna dicionário com chaves 'a', 'b', 'c'.	dict
<code>calculate_lamdapo(self, dimplana)</code>	Calcula o índice de esbeltez λ_{po} de pré-flambagem para um elemento plano. Parâmetro: dimplana em cm.	float
<code>calculate_k(self, enrijecido, tipo, lambdapo, b_plana, c_plana, c_total, nome)</code>	Determina o fator de flambagem k do elemento conforme tipo (AA/AL) e condição de enrijecimento. Lança ValueError se $D/b > 0,8$.	float
<code>calculate_lambdap(self, k, dimplana)</code>	Calcula o índice de esbeltez reduzido λ_p pós-flambagem. Parâmetros: k (adim.), dimplana em cm.	float
<code>calculate_bef(self, dimplana, lambdap, lambdapo, nome, c_plana, enrijecido)</code>	Calcula a largura efetiva (bef) do elemento, considerando se é enrijecido ou não, e corrigindo a contribuição do enrijecedor quando aplicável. Retorna bef em cm.	float
<code>run_calculation(self)</code>	Executa o cálculo completo do MLE para todos os elementos da seção e retorna dicionário com área efetiva, detalhes por elemento e Nrd.	dict

Estrutura do dicionário retornado por run_calculation()

```
{
  "geometria": {
    "area_bruta": [cm²],
    "a_efetiva": [cm²]
  },
  "estabilidade": {
    "chi_global": [kN/cm²],
    "sigma_atuante": [kN/cm²]
  },
  "elementos": [
    {
      "elemento": "k",
      "lambda_p": "b_plana",
      "b_efetiva": "b_inefetiva"
    }
  ],
  "final": {
    "n_rd": [kN],
    "unidade": "kN"
  }
}
```

6. Módulo: flexao_mle.py

Implementa o Método da Largura Efetiva (MLE) para flexão em torno do eixo X, com verificação de flambagem lateral com torção (FLT) e iteração para encontrar o deslocamento da linha neutra efetiva. Requer uma instância de EstabilidadeGlobal.

Construtor

```
FlexaoMLE(perfil_geom, estabilidade_obj, cb: float = 1.0, mxsd: float = 0.0)
```

Parâmetros importados

Parâmetro	Tipo	Unidade	Descrição
perfil_geom	objeto PerfilBase	–	Instância do perfil com propriedades calculadas.
estabilidade_obj	objeto EstabilidadeGlobal	–	Instância de EstabilidadeGlobal já calculada para o perfil.
cb	float	–	Fator de momento uniforme equivalente (Cb). Padrão: 1,0.
mxsd	float	kN·cm	Momento solicitante de cálculo (usado para determinar sinal do fator Cs na seção cartola).

Parâmetros calculados

Parâmetro	Tipo	Unidade	Descrição
chi_flt	float	–	Fator de redução por FLT, calculado por calculate_chi_flt().
me	float	kN·cm	Momento crítico elástico de FLT.
lambdapo	float	–	Índice de esbeltez relativo à FLT.
tensao	float	kN/cm ²	Tensão de referência: chi_flt × fy.
desloc_ln	float	cm	Deslocamento iterativo da linha neutra efetiva (calculado em run_calculation).
wef	float	mm ³	Módulo resistente efetivo (mínimo entre fibras tracionada e comprimida).
wcef	float	mm ³	Módulo resistente efetivo em relação à fibra comprimida.

Métodos

Bloco	Descrição	Retorno
get_dimplanas(self)	Extraí dimensões planas (a, b, c) do perfil em cm.	dict
calculate_chi_flt(self)	Calcula o fator chi_flt de redução por flambagem lateral com torção. Chamado no construtor.	float

Bloco	Descrição	Retorno
<code>calculate_psi(self, desloc_ln, c_plana)</code>	Calcula o gradiente de tensão ψ na alma e tensões nas fibras. Atualiza internamente <code>tensao_s</code> , <code>tensao_i</code> , <code>tensao1</code> , <code>tensao2</code> , <code>tensao1c</code> . Parâmetros em cm.	float (ψ)
<code>calculate_sigma(self, nome)</code>	Retorna a tensão atuante no elemento identificado por nome (string do dicionário <code>TIPOLOGIA_ELEMENTOS</code>).	float (kN/cm ²)
<code>calculate_k(self, enrijecido, tipo, lambdap, b_plana, c_plana, c_total, nome, psi)</code>	Determina o fator k de flambagem local com gradiente de tensão ψ . Para enrijecidos, verifica relação I_s/I_a .	float
<code>calculate_lambdap(self, dimplana, sigma)</code>	Calcula λ_{po} de pré-flambagem para flexão. Parâmetros em cm e kN/cm ² .	float
<code>calculate_lambdap(self, k, dimplana, sigma)</code>	Calcula λ_p pós-flambagem para flexão.	float
<code>calculate_bef(self, dimplana, lambdap, lambdapo, nome, c_plana, enrijecido, psi)</code>	Calcula a largura efetiva b_{ef} do elemento em flexão, considerando gradiente de tensão, posição (sup/inf/alma), enrijecimento e distribuição $b_{ef1}+b_{ef2}$ na alma. Atualiza internamente <code>b_nef_alma</code> e <code>yg_alma</code> para uso posterior na iteração.	float (cm)
<code>calculate_desloc(self, resultados_iteracao)</code>	Calcula o deslocamento da linha neutra efetiva com base nas parcelas inefetivas. Chamado iterativamente em <code>run_calculation</code> .	float (cm)
<code>run_calculation(self)</code>	Executa o cálculo completo com iteração (máx. 50 iterações, tolerância 0,01 cm) e retorna dicionário com <code>Mrd</code> e resultados por elemento.	dict

Estrutura do dicionário retornado por `run_calculation()`

```
{
  "geometria_efetiva": {
    "desloc_ln": [mm],
    "wef": [mm³],
    "wcef": [mm³],
    "iteracoes": {}
  },
  "grafico_tensoes": {
    "sigma_sup": [kN/cm²],
    "sigma_inf": [kN/cm²],
    "sigma1": [kN/cm²],
    "sigma2": [kN/cm²]
  },
  "flambagem_flt": {
    "Me": [kN.cm],
    "lambda_0": [cm],
    "chi_flt": [cm]
  },
  "elementos": [
    {
      "elemento": "k",
      "sigma": [kN/cm²],
      "lambda_p": [cm],
      "b_plana": [cm],
      "b_efetiva": [cm],
      "b_inefetiva": [cm]
    }
  ],
  "final": {
    "mrd1": [kN.cm],
    "mrd2": [kN.cm],
    "m_rd": [kN.cm],
    "unidade": [unidade]
  }
}
```

7. Módulo: `tracao.py`

Implementa a verificação de capacidade de tração conforme a NBR 14762:2010, avaliando escoamento da seção bruta, ruptura líquida fora da ligação e ruptura líquida na ligação.

Construtor

```
CapacidadeTracao(perfil_geom, An0: float, An: float, Ct: float)
```

Parâmetros importados

Parâmetro	Tipo	Unidade	Descrição
perfil_geom	objeto PerfilBase	–	Instância de USimples, UEnrijecido ou Cartola já com propriedades calculadas.
fy	float	kN/cm ²	Tensão de escoamento (convertida de MPa).
fu	float	kN/cm ²	Tensão de ruptura (convertida de MPa).
A	float	cm ²	Área bruta da seção (convertida de mm ²).
An0	float	mm ²	Área líquida da seção fora da região de ligação (furos, etc.).
An	float	mm ²	Área líquida na região de ligação.
Ct	float	–	Fator de redução por shear lag ($0 < Ct \leq 1,0$).

Parâmetros calculados

Parâmetro	Tipo	Unidade	Descrição
ntrd	float	kN	Força axial resistente de cálculo (mínimo das três parcelas).

Métodos

Função	Descrição	Retorno
<code>run_calculation(self)</code>	Calcula as três parcelas de resistência à tração e retorna dicionário com resultados.	dict

Estrutura do dicionário retornado por `run_calculation()`

```
{
  "geometria": {
    "area_bruta": "area_liquida_fora",
    "area_liquida_ligacao": "area_liquida_ligacao"
  },
  "parcelas": {
    "ntrd_bruta": "ntrd_liq_fora",
    "ntrd_liq_lig": "ntrd_liq_lig"
  },
  "final": {
    "n_rd": float [kN],
    "unidade": "kN"
  }
}
```

8. Módulo: cortante.py

Calcula a resistência ao esforço cortante da alma conforme a NBR 14762:2010, verificando os três domínios. Para o perfil Cartola, multiplica a resistência por 2 (quantidade de almas resistentes).

Construtor

```
ResistenciaCortante(perfil_geom)
```

Parâmetros importados

Parâmetro	Tipo	Unidade	Descrição
perfil_geom	objeto PerfilBase	—	Instância do perfil com propriedades calculadas.
E	float	MPa	Módulo de elasticidade: 200 000 MPa.
fy	float	kN/cm ²	Tensão de escoamento (convertida de MPa).
t	float	cm	Espessura da chapa (convertida de mm).

Métodos

Bloco	Descrição	Retorno
run_calculation(self)	Calcula Vrd com base na relação h/t e nos limites de domínio. Retorna dicionário com relação h/t, limites, número de almas e Vrd.	dict

Estrutura do dicionário retornado por run_calculation()

```
{
  "h_t_relacao": float,
  "limite_1": float, "limite_2": float,
  "almas_consideradas": int (1 ou 2),
  "final": {"v_rd": [kN], "unidade": "kN"}
}
```

9. Módulo: combinados.py

Fornece métodos estáticos para verificação de esforços combinados. Não requer instanciação – os métodos são chamados diretamente pela classe.

Métodos

Função	Descrição	Retorno
EsforçosCombinados.flexao_composta(nsd, nrd, mxsd, mxrd)	Verifica a interação $(N_{sd}/N_{rd}) + (M_{xsd}/M_{xrd}) \leq 1,0$ (tração ou compressão + flexão em X). Todos os parâmetros em kN ou kN·cm.	dict
EsforçosCombinados.flexao_cortante(mxsd, mxrd, vsd, vrd)	Verifica a interação $(M_{xsd}/M_{xrd})^2 + (V_{sd}/V_{rd})^2 \leq 1,0$. Parâmetros em kN·cm e kN.	dict

Estrutura dos dicionários retornados

```
{
  "verificacao": str (descrição da verificação),
  "nsd"/"mxsd"/"vsd": float (esforços solicitantes),
  "nrd"/"mxrd"/"vrd": float (esforços resistentes),
  "ratio": float (valor da interação),
  "aprovado": bool,
  "mensagem": "ATENDIDA" | "NÃO ATENDIDA"
}
```

10. Sistema de Avisos (perfil.aviso)

As classes de perfil acumulam mensagens de aviso em `perfil.aviso` (`list[str]`). Esses avisos são adicionados pelas classes de verificação quando relações b/t ultrapassam limites normativos ou quando parâmetros auxiliares atingem valores limítrofes. Nenhum aviso interrompe o cálculo; o usuário deve verificar a lista após a execução.

```
if perfil.aviso:
    for msg in perfil.aviso:
        print("AVISO:", msg)
```

11. Fluxo de uso recomendado

A sequência abaixo deve ser seguida para garantir que as dependências entre classes sejam satisfeitas:

```
# 1. Instanciar o perfil (fornece todas as propriedades geométricas)
perfil = UEnrijecido(t=2.0, fy=345, bw=200, bf=75, d=20)
```

```
# 2. Instanciar EstabilidadeGlobal (calcula chi_global)
estab = EstabilidadeGlobal(perfil, lx=3000, ly=3000, lz=3000)
```

```
# 3. Capacidade à compressão
comp = CompressaoMLE(perfil, chi=estab.chi_global)
resultado_comp = comp.run_calculation()
```

```
# 4. Capacidade à flexão
flex = FlexaoMLE(perfil, estab, cb=1.0, mxsd=500.0)
resultado_flex = flex.run_calculation()
```

```
# 5. Resistência ao cortante
cort = ResistenciaCortante(perfil)
resultado_cort = cort.run_calculation()
```

```
# 6. Tração (quando aplicável)
trac = CapacidadeTracao(perfil, An0=350.0, An=350.0, Ct=0.9)
resultado_trac = trac.run_calculation()
```

```
# 7. Esforços combinados
EsforçosCombinados.flexao_composta(nsd=50, nrd=resultado_comp['final']['n_rd'],
                                     mxsd=300, mxrd=resultado_flex['final']['m_rd'])
```

